

二、系统设计

詹文翰 高级实验师

zhanwenhan@163. com

面向对象的软件设计

需求分析过程 —————→ “做什么”

软件设计过程 —————→ “怎么做”

软件设计是一个**迭代**的过程，通过设计，需求被变换为构建软件的“蓝图”，即将软件分析模型中使用功能和行为来展示的需求信息转化为**体系结构设计、数据或者类设计、接口设计和构件级设计**等。

所有系统均可表示为两个方面：**静态结构**和**动态行为**。因此，面向对象的软件设计就是需要建立系统的**静态模型**和**动态模型**。

静态建模

静态模型描述系统的**静态结构**，是系统建模的**基础**，能够说明**系统包含什么以及它们之间的关系**，但并不解释系统中各个对象是如何协作来实现系统功能的。

UML中典型的静态建模工具：**类图**、**对象图**。

类图

类图是面向对象建模的主要组成部分，表示模型中的**类**、**类的内部结构**以及**类之间的关系**。其既可以用于程序的系统概念建模，也可以用于详细建模。

可以通过类图直接将模型转换成编程代码（**正向工程**）；也可以基于现有代码生成对应类图（**逆向工程**），用于辅助代码分析。

类的表示

类的表示:

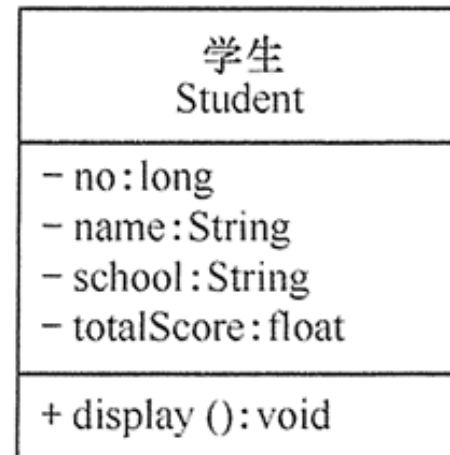
(1) **类名 (Name)** 是一个字符串。

(2) **属性 (Attribute)** 是指类的特性, 即类的成员变量。

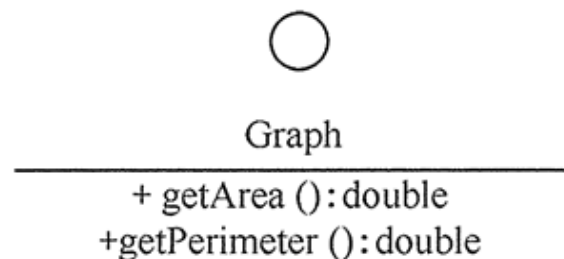
[可见性]属性名:类型[=默认值]

(3) **操作 (Operations)** 是类的任意一个实例对象都可以使用的行为, 是类的成员方法。

[可见性]名称(参数列表)[:返回类型]



接口 (抽象基类) 的表示:



类间关系

根据类与类之间的耦合度从弱到强排列，UML 中的类图有以下几种关系：依赖关系、关联关系、聚合关系、组合关系、泛化（实现）关系。

类间关系

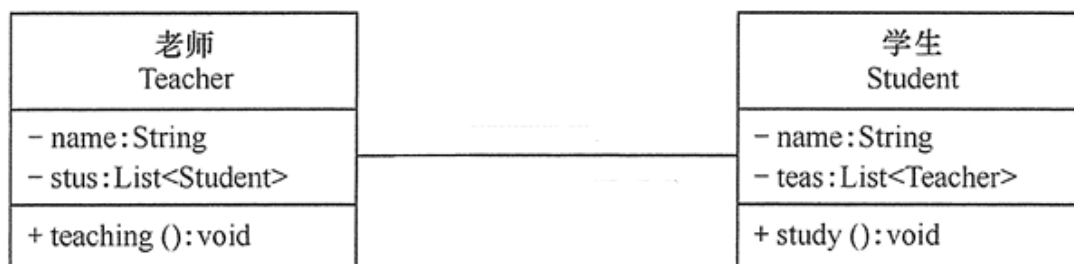
根据类与类之间的耦合度从弱到强排列，UML 中的类图有以下几种关系：依赖关系、关联关系、聚合关系、组合关系、泛化（实现）关系。

依赖（Dependency）关系是一种临时性的关联，大多表示“使用”。在代码中，某个类的方法通过局部变量、方法的参数或者对静态方法的调用来访问另一个类（被依赖类）中的某些方法来完成一些职责。

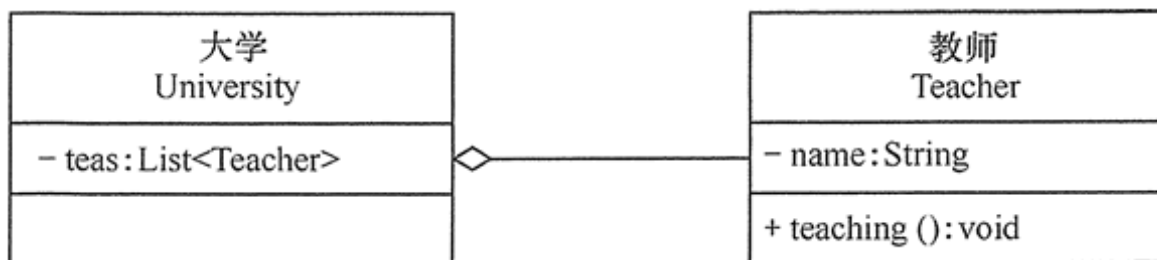


类间关系

关联（Association）关系是对象之间的一种引用关系，用于表示一类对象与另一类对象之间的联系，如老师和学生、师傅和徒弟、丈夫和妻子等。

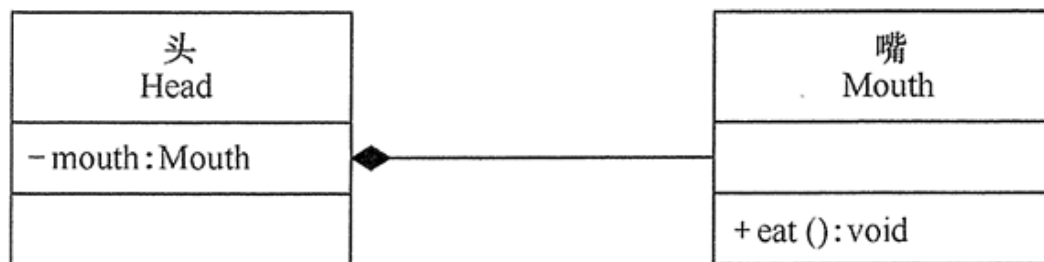


聚合（Aggregation）关系是关联关系的一种，是强关联关系，是整体和部分之间的关系，是 **has-a** 的关系。

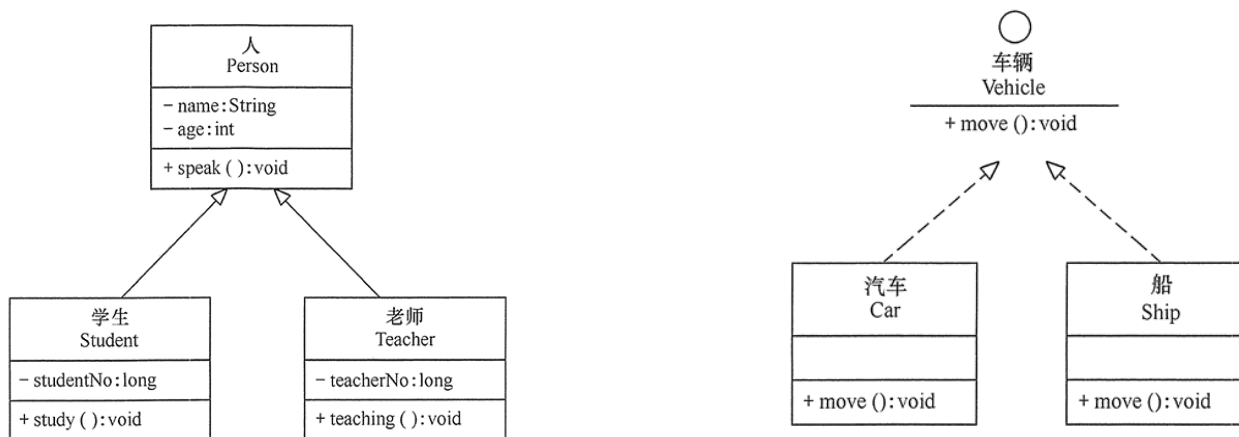


类间关系

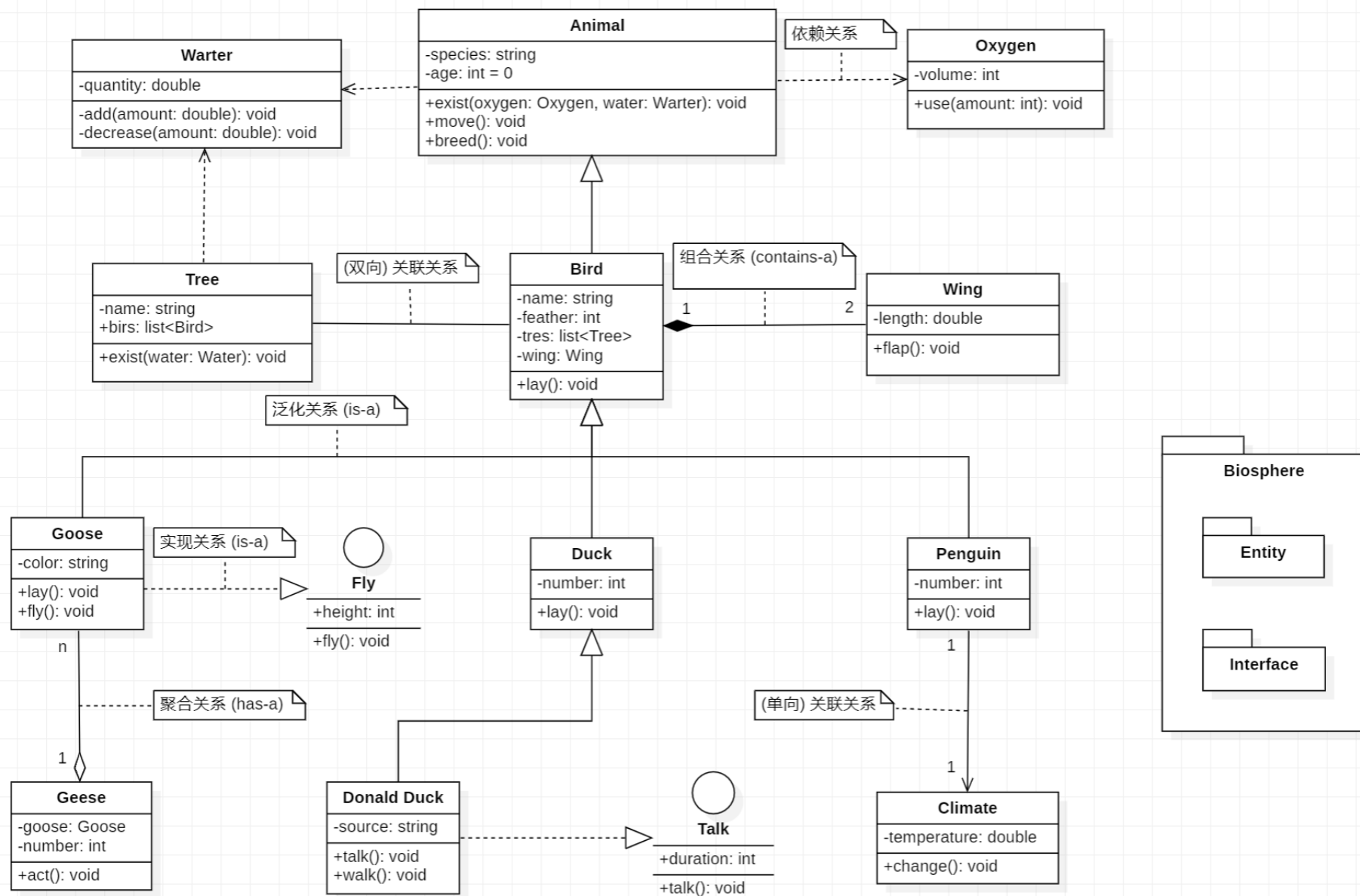
组合（Composition）关系也是关联关系的一种，也表示类之间的整体与部分的关系，但它是一种更强烈的聚合关系，是 **contains-a** 关系。



泛化（Generalization）关系或实现（**Realization**）关系是对象之间耦合度最大的一种关系，表示一般与特殊的关系，是父类与子类之间的关系，是一种继承关系，是 **is-a** 的关系。



类图举例



动态建模

任何系统功能均是通过系统**结构元素之间的交互**来完成的。动态建模描述了使用**系统功能的具体过程**以及各对象间的**交互关系**。结构元素（对象）之间通过**通信协作**或者**自身状态改变**的方式来实现系统的功能。

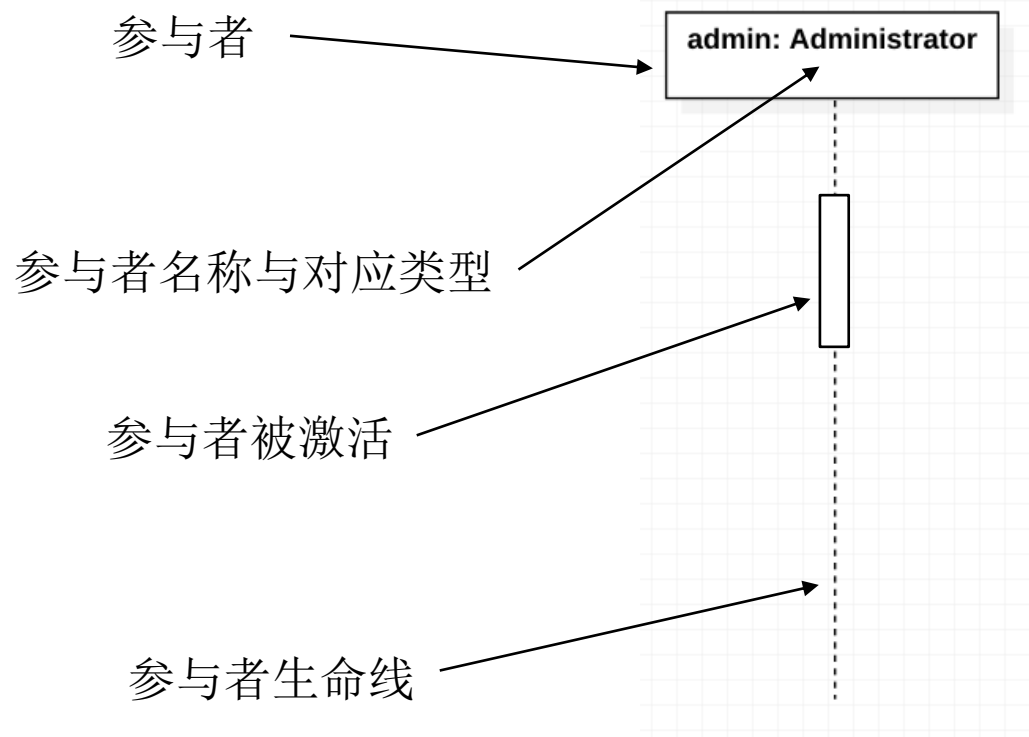
UML中典型的动态建模工具：**交互图（顺序图、时序图、通信图）、状态机图、活动图（需求分析）**。

顺序图（序列图）

顺序图用于描述执行特定用例时会触发哪些系统结构元素之间的交互，以及这些交互是以何种次序发生的。

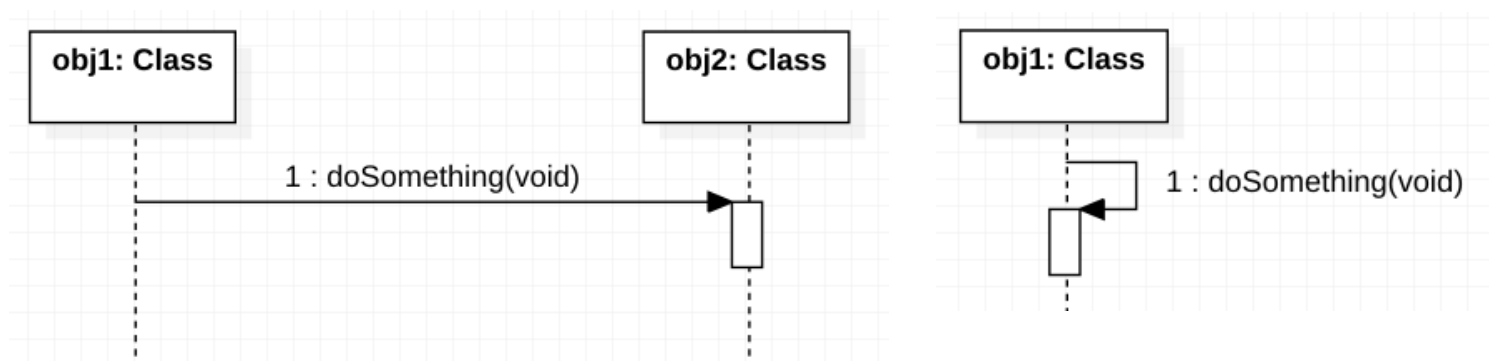
顺序图由参与者和消息构成。

参与者表示

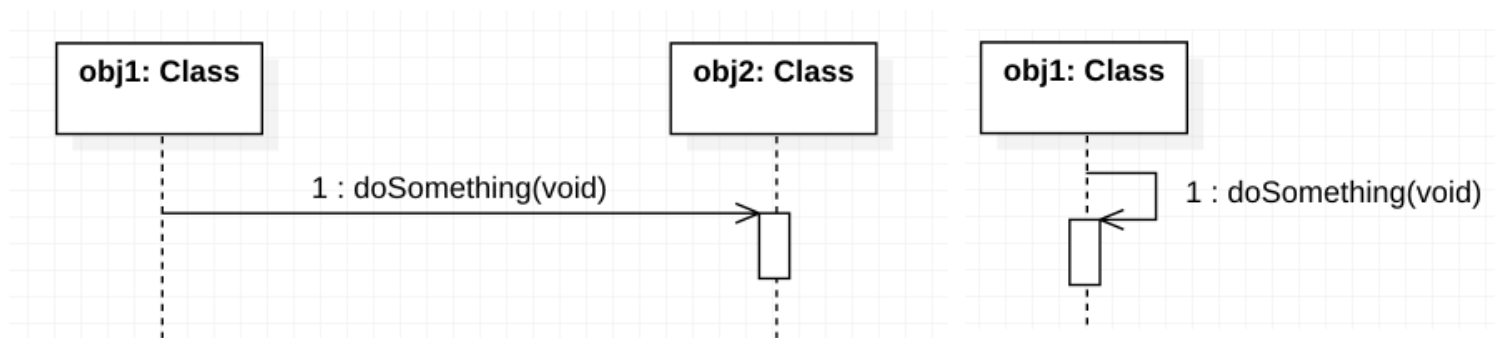


消息表示

同步消息：

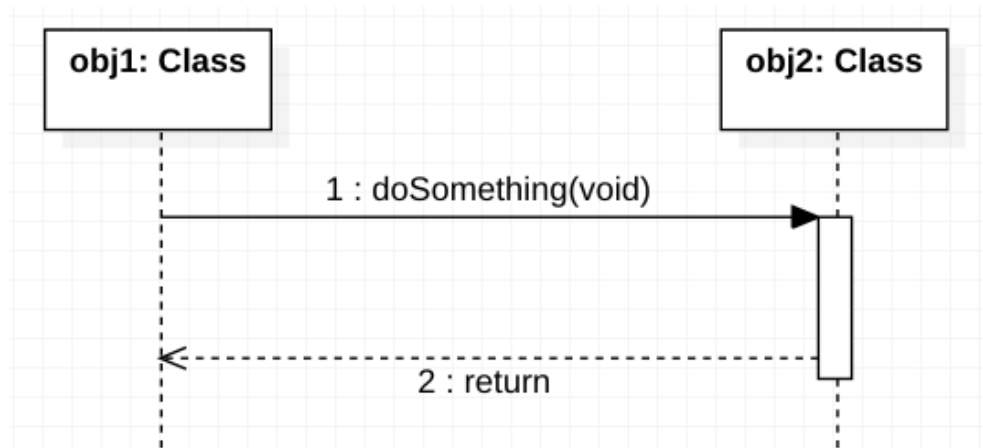


异步消息：



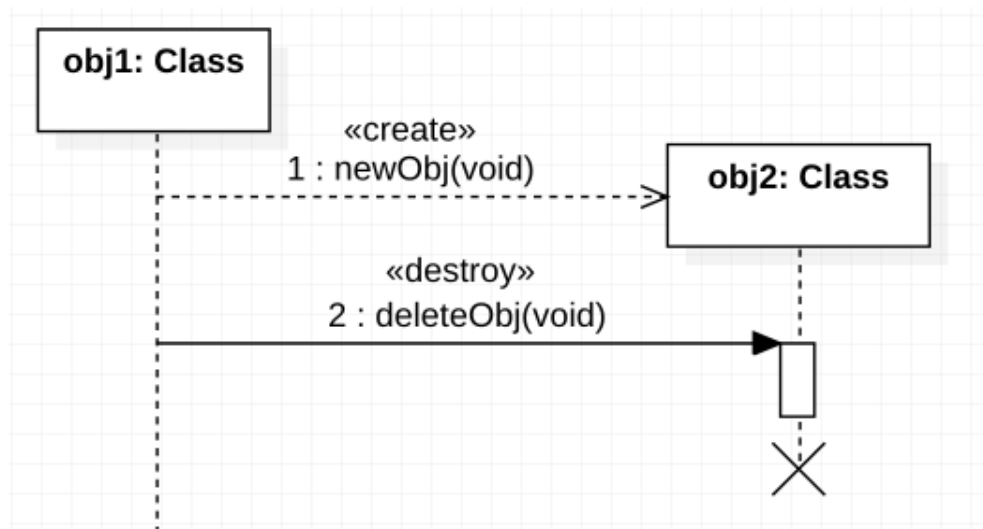
消息表示

返回消息:



参与者创建消息:

参与者销毁消息:

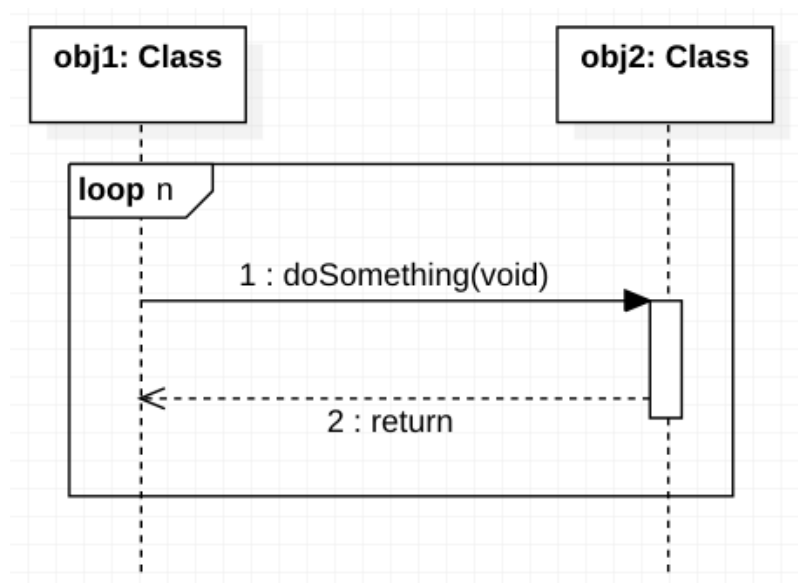


顺序片段

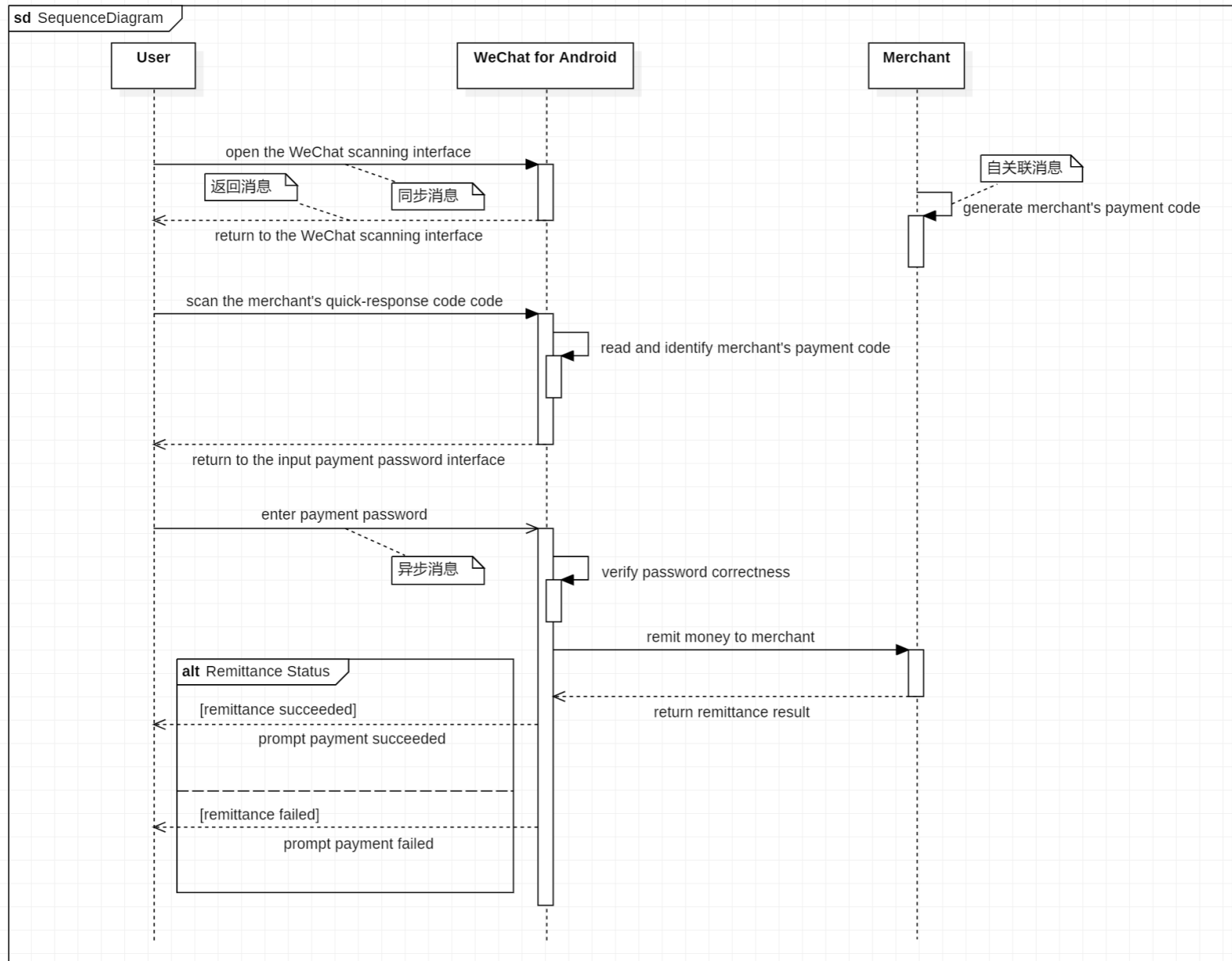
顺序片段可以让顺序图表达出复杂的过程逻辑。

常用的顺序片段包括：

片段类型	参数	作用
ref	无	引用其他片段
loop	循环条件	循环执行片段
alt	多选一	if-else选择
opt	可选	if选择



顺序图举例



物理建模

静态建模和动态建模是按照**逻辑观点**对应用领域的**概念建模**，而物理建模则是对应用自身的**实现结构建模**，提供了将系统中的类映射成物理构件和节点的机制。

UML中典型的物理建模工具：**构件图**（组件图）、部署图（分布式系统）。

构件图

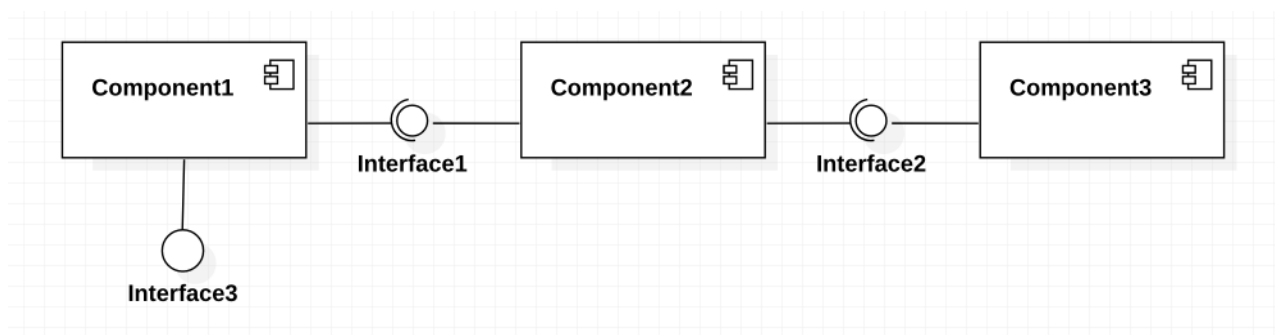
UML规范将构件（组件）定义为“系统中的一个定型化的、可配置的和可替代的部件，该部件封装并暴露一系列接口”。最常见的构建表现形式：动态库和静态库。

构件图（组件图）描述系统中的**构件**、**接口**及构件之间的**依赖**关系，帮助用户理清构件之间的相互关系，便于利用这些依赖关系来评估构件的修改对系统所造成的影响。

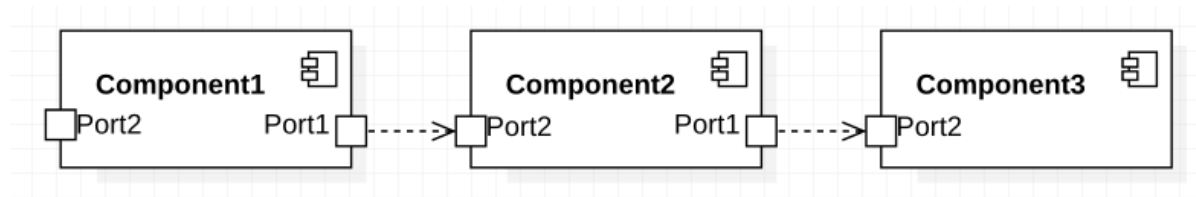
构件图表示法

构件图的表示方法还不是很统一。

第一种表示法：



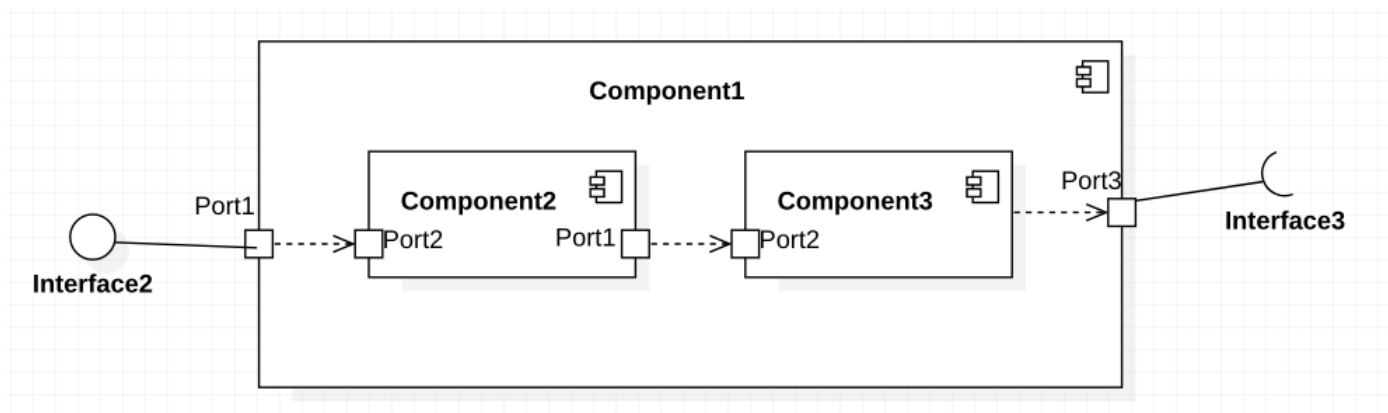
第二种表示法：



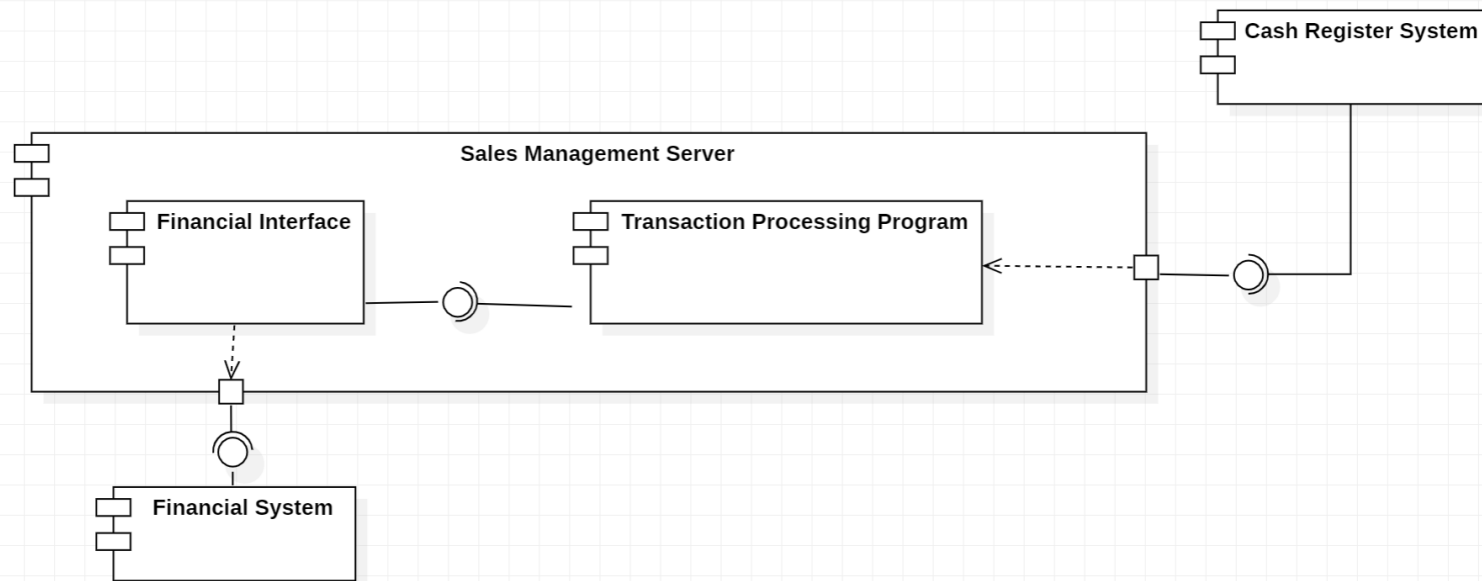
黑箱构件图与白箱构件图

上面的例子均表现为黑箱视图，其仅显示构件的外观，不指定构件内部实现。黑箱视图擅长显示系统的整体视图和构件依赖关系，通常包含多个组件。

白箱视图通常用于显示构件的内部实现，说明其由哪些类、接口和其它包含组件组成。白箱视图通常关注一个组件的内容。



构件图举例



构件图说明

1、构建划分原则应该是什么样的？

- 高内聚
- 松耦合

2、构件图应该在何时使用？

- 在系统概要设计时作为一个系统功能的初步划分
- 在系统详细设计时作为一个逻辑相对独立的程序或系统模块

实验环节

完成以下实验内容：

- 完成实验指导书中指定练习
 - 无需提交实验报告
- 完成本课程项目的需求分析
 - 绘制本项目类图并撰写对应说明
 - 绘制本项目顺序图并撰写对应说明
 - 绘制本项目构件图并撰写对应说明